Silent Leaks: Harvesting Secrets from Shared Linux Environments

Ionut Cernica

UiPath

Table Of Contents

whoami

UiPath

Application Security Engineer at UiPath

AI Security Researcher

Bug Bounty Hunter

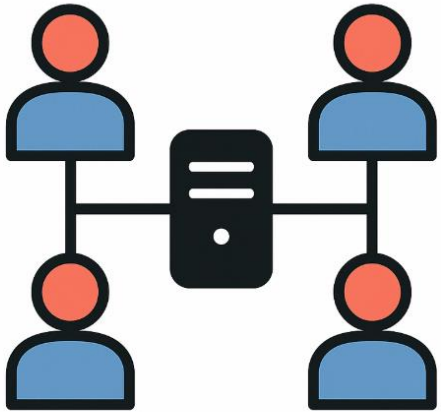
Former CTF Player

Former Entrepreneur

Multi-user Linux environments

Web Hosting Panels

Dev servers, education labs, VPS providers, CTF infra



Assumptions

No root, no local privilege escalation

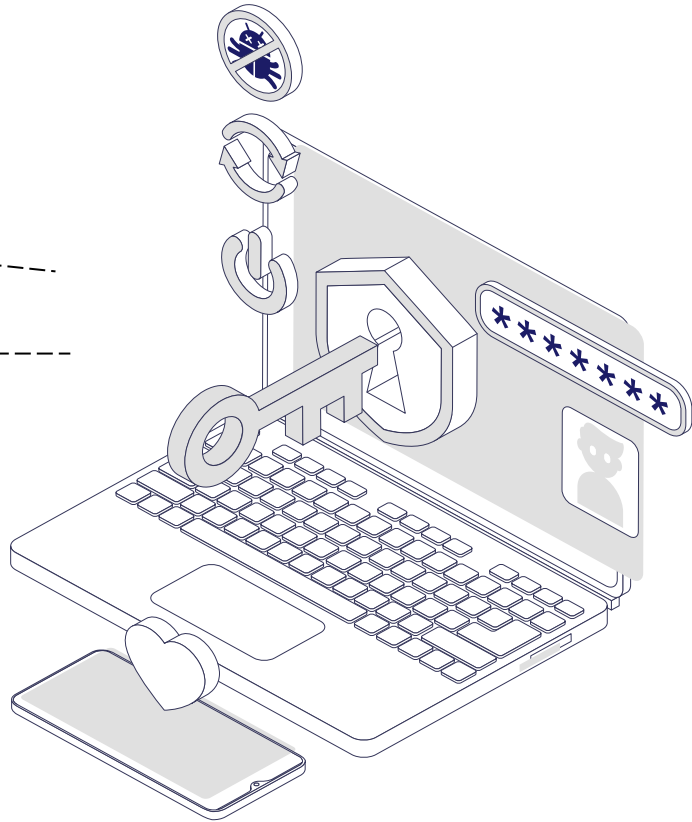
Just a basic user with shell access (or web shell)

Attacker Goals

Discover secrets (DB creds, API keys)

Fingerprint other users/domains

Prepare for lateral movement



Process Argument Leaks

Linux process visibility 101

cat /proc/[pid]/cmdline

ps auxww

pgrep

Why is this default behavior?

🐧 **Legacy of transparency:** Linux was designed for trusted multi-user environments (e.g., universities, shared labs)

🧑‍💻 **Debugging & monitoring:** Useful for sysadmins and developers to inspect misbehaving processes

🔄 **Backward compatibility:** Changing this behavior now would break many tools and workflows

About Disclosure

I will not name affected providers in this presentation.

The issue was reported in early April to all major web hosting platforms.

One provider granted permission to demonstrate the issue live, but not to publish their name on slides.

Fixes are in progress, but may take time to propagate across all customer environments.

To act responsibly, I've chosen not to name providers until widespread remediation is confirmed.

Let’s run ‘ps auxww’ linux command in a loop and explore some real-world leakage scenarios

Real world cases (ps auxww)

t3st1.i+ 88251 0.0 0.0 5744 1116 ? S 13:21 0:00 timeout 60 /opt/[REDACTED]/php/8.3/bin/php -d safe_mode=off -d display_errors=on -d opcache.enable_cli=off -d open_basedir= -d error_reporting=341 -d max_execution_time=60 -c /var/www/vhosts/system/t3st1.io/etc/php.ini [REDACTED]/wp-toolkit/vendor/wp-cli/wpt-wp-cli.php --path=/var/www/vhosts/t3st1.io/httpdocs --no-color config set **DB_USER 'wp_new_user'** --raw

t3st1.i+ 81447 0.0 2.2 127080 44172 ? R 13:18 0:00 /opt/[REDACTED]/php/8.3/bin/php -d safe_mode=off -d display_errors=on -d opcache.enable_cli=off -d open_basedir= -d error_reporting=341 -d max_execution_time=60 -c /var/www/vhosts/system/t3st1.io/etc/php.ini [REDACTED]/wp-toolkit/vendor/wp-cli/wpt-wp-cli.php --path=/var/www/vhosts/t3st1.io/httpdocs --no-color config set **DB_PASSWORD 'T3sting123!!'** --raw

Real world cases

root 126598 0.0 0.1 6972 3372 ? S 12:05 0:00 sh -c /usr/sbin/useradd **t3st2** -d /home/t3st2;echo -e '**T3sting123! T3sting123!**'|/usr/bin/passwd t3st2;mkdir /home/t3st2/public_html;chown -R t3st2:t3st2 /home/t3st2/public_html;chmod 711 /home/t3st2

Real world cases

root 126598 0.0 0.1 6972 3372 ? S 10:01 0:00 /bin/bash -c /www/server/mysql/bin/mysql -u root -p88dd296e2086da6e t3st333_t3st1w_io < /tmp/vdRouPtcOisEGIBq/YFQKZmYnVzkbAhxS.sql

Real world cases

t3st1 8038 55.0 3.4 204660 69824 ? Ssl 20:34 0:00 php /usr/local/bin/wp core install --url=t3st1.io --title=t3st1.io --admin_user=t3st1_io --admin_password=T3sting123! --admin_emai=[REDACTED]@gmail.com --skip-email --path=/var/www/t3st1/data/www/t3st1.io

Real world cases

root 8007 0.0 0.1 5672 3552 ? S 21:34 0:00 su -s /bin/bash -l -c /usr/local/bin/wp core config --dbname='t3st1_io' --dbuser='t3st1_io' --dbpass='eMd6LCwY6WMXVLh8' --dbhost='localhost' --path=/var/www/t3st1/data/www/t3st1.io - t3st1

Real world cases

root php /usr/bin/wp config create --dbhost=127.0.0.1:3306 --dbname=t3st3 --dbuser=t3st3 --dbpass=t0ecpBDfp33x5In5PETA --locale=en_U

Solutions to Restrict Process Info Leaks

Approach	Description	Drawbacks
`hidepid` option on `/proc` mount	Limits visibility of other users' processes: `mount -o remount,hidepid=2 /proc`	May break monitoring/debugging tools
Use containers (e.g., Docker, LXC)	Containers have their own `/proc` namespaces	Requires containerized environment
Enable `CageFS`, `Virtuozzo`, etc.	File system-level user isolation (used by hosting providers)	Adds complexity, mostly for shared hosting

Jailing and Isolation Systems in Shared Environments

System	Description
CageFS	Filesystem-level isolation used in shared hosting. Hides `/proc`, `/etc/passwd`, and other sensitive paths from users.
chroot	Changes the apparent root directory for a user/process. Simple, but easy to escape if not properly configured.
Linux namespaces	Used in containers. Provide isolation of processes, users, networks, and more. Foundation of Docker and LXC.
Firejail	Lightweight sandbox tool using namespaces and seccomp. Can restrict file access and syscalls.
Virtuozzo/OpenVZ	Container-based virtualization for isolating environments with lower overhead than full VMs.
AppArmor/SELinux	Mandatory Access Control (MAC) systems that restrict what processes can do or access, even as root.

Bug Bounty Time

🚨 Escaping the Cage: Bypassing CageFS in a Major Hosting Panel

I tested a popular web hosting panel configured with CageFS.

CageFS is designed to restrict users to their own virtualized environment.

I break out of the jail...

The escape was possible by leveraging a binary provided by the hosting panel itself.

This binary was inadvertently running outside of CageFS, exposing the real host environment.

🚨 Escaping the chroot in a Major Hosting Panel

I tested a popular web hosting panel configured with chroot.

They used filebrowser from another chroot

I took Remote Command Execution on filebrowser (undocumented command, I found it in the source code)

I escaped chroot from there

🚨 Credential Exposure via LiteSpeed in Jailed Environments

The most important hosting environments I tested was using LiteSpeed.

I discovered a critical issue:
- By executing a script and reading from /proc/self/fd/2,
- I could access the shared stderr.log used by all users on the system.

This allowed me to leak error output from other users' scripts in real time.

The LiteSpeed Team fixed this bug.
- On 31 March 2025 I reported this
- 3 April 2025 they confirmed me the fix

Examples of stderr.log data leak in isolated environments

You could find full requests and their responses: and I reported bearer tokens, user/password from post requests and active cookies of users:

Paypal api token:
Authorization: Basic QkFBbnd**[REDACTED]**VVCQQ==

Cookies:
Set-Cookie: JSESSIONID=A11583633[REDACTED]C1BB1A617.[REDACTED]-NODEV4;

Credentials:
username=[REDACTED]@[REDACTED].com&password=[REDACTED]

Temp File Leaks & Poor File Permissions

Many scripts generate sensitive files in /tmp directory: SQL dumps, logs, credentials, even .php files with hardcoded secrets.

It's possible to write a script that monitors /tmp and grabs newly created files in real-time.

The script should run in an infinite loop, as some files may be deleted immediately after use.

Examples

I was able to read the /tmp/[REDACTED]/var/log/*" and there is the install log of a Web Hosting Panel

[INFO] Your Mailman password is: **XdVLFyaY7Zyj**
[INFO] Your [REDACTED] admin password is: **yGZ7RBwhvNHA4ZJ**
[INFO] Your MySQL root password is: **Kr4SdvHyTSoqbvqjYe7c**

Examples

/tmp/vdRouPtcOisEGlBq/YFQKZmYnVzkbAhxS.sql

This temporary file was created to restore a database, then deleted shortly after

The entire lifecycle, creation, usage and deletion can happen within milliseconds

Examples – bug bounty

Even in a "hardened" environment…
- The system was isolated
- /proc/ was not accessible
- ps command was unavailable

BUT…

I found other user install scripts running from:
/tmp/dNtccKKZSr/main.php

The file was world-readable, exposing credentials


KEEP
CALM
IT'S
DEMO
TIME

Closing Remarks

- **Multi-user Linux systems are full of quiet leaks and not always obvious, but often devastating**
- **Security isn’t just about exploits it’s about assumptions**
- **Tools like ps, pgrep are trusted because they’re standard but in shared environments, they become silent recon tools**
- **These leaks aren’t flashy, but they’re persistent, stealthy, and often overlooked by both defenders and auditors**

Thanks

Do you have any questions?

ionut.cernica@gmail.com